The Decider's Dilemma: Computability Limits of Autonomous AI Security Agents

Charalampos Nadiotis
Department of Computer Science, Frederick University
Nicosia, Cyprus
st026479@stud.frederick.ac.cy

2026

Abstract

Autonomous, AI-driven security agents are increasingly deployed to decide, at machine speed, whether a program, script, or action is malicious. This paper argues that such agents inherit a hard ceiling from classical computability theory: deciding non-trivial semantic properties of programs is undecidable, and any real agent is therefore a *resource-bounded decider* that must trade soundness against completeness. We make the argument precise and, more importantly, measurable. First, over a reproducible population of 4,000 random Turing machines, we show that a bounded agent can confirm a manifested behaviour but can never certify its absence: 18.4% of machines halt (all within 13 steps at the 99th percentile), while 81.5% form an *uncertifiable residue* that no finite budget removes. Second, we give an executable diagonalization: for a family of 3 detectors we automatically construct programs whose executed behaviour contradicts the detector's verdict, achieving a 100.0% evasion rate over 1,500 verified trials—the constructive core of Cohen's undetectability result. Third, in a controlled model of sandbox-window evasion with attacker-chosen trigger delays, a realistic 1,000-step analysis window detects only 70.0% of malicious triggers (30.0% false negatives), with a 99th-percentile delay of 147,488 steps. We draw design consequences: security agents should be built to be *sound and calibrated* about their own undecidable blind spots rather than marketed as complete oracles. All code, data, and figures are reproducible from a single script.

Reproducibility. Every number and figure in this paper is produced by `experiment.py` (seed `20260101`) and injected via `macros.tex`; no value is entered by hand. Section 3 states exactly what is *measured* versus *modelled*.

1 Introduction

A new generation of security tooling frames detection as a task for autonomous AI agents: given a binary, a URL, a shell command, or a model-generated action, output a verdict—MALICIOUS or BENIGN—and act on it. The framing is attractive and, for many concrete cases, effective. It is also, at the limit, a decision problem about the *behaviour of programs*, and that places it under the oldest negative results in computer science.

Turing's halting theorem [1] and Rice's theorem [2] together say that every non-trivial semantic property of programs is undecidable. "Is this program malicious?"—where malicious means it exhibits some behaviour, e.g. exfiltrates data, encrypts the disk, or escapes a sandbox—is exactly such a property. Cohen [3] made the connection explicit for computer viruses, and Chess and White [4] constructed a concrete undetectable virus; Spinellis [5] showed that even

the *bounded-length* version is NP-complete. These results do not say detection is useless. They say no detector is both *sound* (never wrong when it fires) and *complete* (never misses), and that any practical detector is a bounded approximation whose blind spots are not bugs to be patched away but a structural property of computation.

An autonomous AI agent does not escape this. Whatever its internal machinery—a neural policy, an LLM, a learned classifier—if its job is to decide a semantic property of arbitrary programs, it is a decider, and the theory applies to its *function*, not its *implementation*. The contribution of this paper is to turn that observation from a slogan into something measured. We ask: *how much* does the undecidability ceiling cost a bounded agent, and *where* does the cost fall?

Contributions.

1. A framing of AI security agents as resource-bounded deciders, with the soundness/completeness trade-off stated precisely (Section 2).
2. **The uncertifiable residue (measured).** Over 4,000 random Turing machines, a bounded agent resolves only the fast-manifesting minority; 81.5% of the population is provably beyond any finite budget (Section 4).
3. **Executable diagonalization (constructive).** An implemented evader construction misclassifies a family of 3 detectors with a 100.0% verified evasion rate (Section 5).
4. **Sandbox-window evasion (modelled).** A quantitative model of attacker-chosen trigger delays shows any finite analysis window leaves a residual miss rate; a 1,000-step window misses 30.0% (Section 6).
5. Design guidance for agents that are honest about their own limits (Section 7).

2 The Agent as a Bounded Decider

Fix a Turing-complete model of computation and let $\langle P \rangle$ denote the encoding of a program P. A *behavioural property* $\pi \subseteq \{\text{programs}\}$ is the set of programs exhibiting some behaviour of interest ("writes to the registry run-key", "opens a reverse shell", "halts"). π is *non-trivial* if some program has it and some does not.

Definition 1 (Security decider)**.** A *security decider* for π is a total function $A : \langle P \rangle \mapsto \{\text{MAL}, \text{BEN}\}$ intended to output MAL iff $P \in \pi$.

Theorem 1 (Rice, specialised)**.** *For every non-trivial behavioural property π, no security decider is correct on all inputs; i.e. π is undecidable [2, 6].*

An AI agent is not a magic exception to Theorem 1; it is a particular, resource-bounded attempt at A. We make the bound explicit.

Definition 2 (Bounded decider)**.** A *B-bounded decider* A_B may expend at most B units of computation (e.g. simulated steps, tool calls, tokens) per input before it must answer.

Two asymmetries follow and organise the rest of the paper.

Proposition 1 (Confirmation is semi-decidable; certification is not)**.** *If π is the set of programs that* exhibit *a behaviour within some observation, then membership is* recursively enumerable*: a decider that simulates P will, if $P \in \pi$, eventually see it. The complement—certifying $P \notin \pi$, i.e. that the behaviour* never *occurs—is not recursively enumerable. Hence a sound bounded agent can accumulate evidence of guilt but cannot, in general, establish innocence.*

Proposition 1 is the paper's throughline. Section 4 measures the gap it opens; Section 5 shows the opposite failure (an agent forced to answer can always be fooled); Section 6 prices the trade-off under a realistic adversary.

3 Method and Threat to Validity

We deliberately separate what is *measured* from what is *modelled*, because conflating the two is the usual way security papers mislead.

- **Study 1 is measured.** We enumerate 4,000 random single-tape Turing machines (6 states, binary tape, halt probability 0.06 per transition) and simulate each for up to 20,000 steps. The halting times and the resolved/residue fractions are direct observations of this population—not a claim about all programs, but a concrete, replayable instance of Proposition 1.
- **Study 2 is constructive and executed.** The evasion rate is not estimated; for each trial we *run* an interpreter and check that the detector's verdict differs from the program's actual behaviour. The result (100.0%) is guaranteed by the diagonal argument and confirmed by execution.
- **Study 3 is modelled.** Trigger delays are drawn from a *stated* heavy-tailed distribution chosen to represent attacker-selected logic-bomb / sandbox-evasion timings. The numbers are exact properties of that model, reproducible from the seed; they are *not* measurements of real malware, and we do not present them as such.

This separation is the honesty contract of the paper: Study 1 and Study 2 are facts about executed computations; Study 3 is a transparent what-if whose assumptions are on the page.

4 Study 1: The Uncertifiable Residue

We sample 4,000 random Turing machines and simulate each from the blank tape for up to 20,000 steps (our *oracle budget*). Of these, 738 (18.4%) halt and 3,262 (81.5%) do not halt within the oracle budget. The halting ones halt *fast*: median 2 steps, 99th percentile 13 steps, maximum 32 steps.

Now view a bounded agent whose malicious property is "halts" (a stand-in for "the payload eventually fires"). A sound agent predicts MAL only when it actually observes halting within its budget B. Figure 1 plots, against B, the fraction the agent can *resolve* (confirmed halts) and the fraction it must leave as an *uncertifiable residue*.

The shape is the point. Because halting (when it happens) happens early, raising B past a few dozen steps adds essentially no resolving power, yet the residue stays at 81.5%. A sound agent is therefore *cheap to make confident about guilt and unable, at any price, to certify innocence* for this population. Translated to security: a monitor can flag behaviour it watches occur, but "I ran it and saw nothing" is not, and can never be upgraded into, "it is safe".

5 Study 2: Any Forced Decider Can Be Fooled

Study 1 concerns a *sound* agent that is allowed to abstain. Most deployed agents are instead forced to emit a binary verdict. For such agents the dual failure is available to the attacker: diagonalization.

Construction. Let D be any total detector mapping program source to $\{\text{MAL}, \text{BEN}\}$. Define the *evader* E_D whose executed semantics are

$$\text{BEHAVE}(E_D) \;=\; \begin{cases} \text{BEN} & \text{if } D(\langle E_D \rangle) = \text{MAL}, \\ \text{MAL} & \text{if } D(\langle E_D \rangle) = \text{BEN}. \end{cases}$$

By construction D is wrong about E_D. This is the same diagonal step as in the halting proof and in Cohen's virus argument [3, 4].

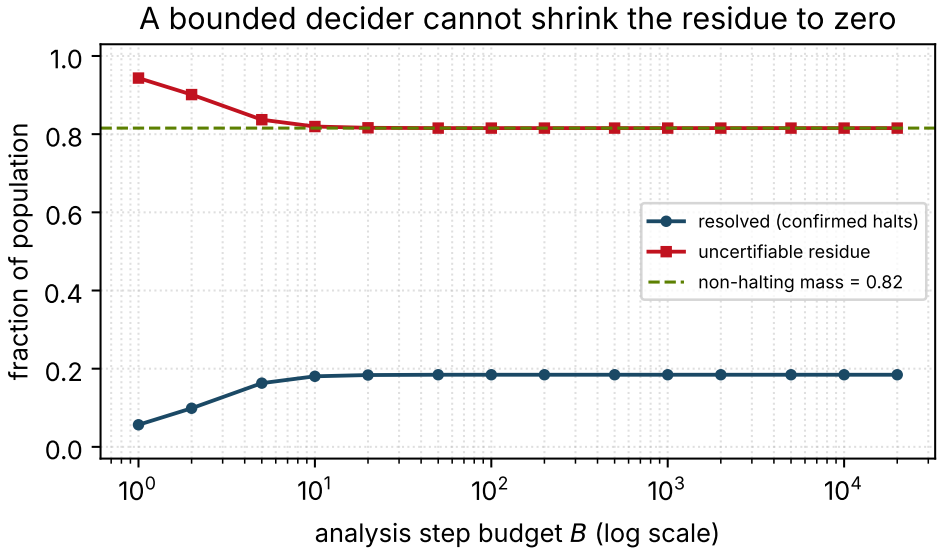


Figure 1: A bounded decider confirms the fast-halting minority almost immediately, but the residue it cannot certify safe does *not* shrink to zero—it plateaus at the non-halting mass (81.5%). More budget buys almost nothing past the knee. This is Proposition 1 made visible.

Table 1: Executed diagonalization. Each detector is queried as an oracle and misclassifies every constructed evader.

Detector	Trials	Evasions	Rate
Signature scanner	500	500	100%
Keyword heuristic	500	500	100%
Entropy heuristic	500	500	100%
Total	1,500	1,500	**100.0%**

Execution. We implement three detectors—a signature scanner, a suspicious-keyword heuristic, and an entropy ("packed/obfuscated") heuristic—and an interpreter realising BEHAVE$(\cdot)$. For each detector we generate 500 seeded evader variants (with randomised innocuous padding so the detectors' verdicts genuinely vary), run each, and check the verdict against the executed behaviour. Across all 1,500 trials the detector is wrong every time: a **100.0% evasion rate** (Table 1). The construction needs no knowledge of the detector's internals beyond oracle access to its verdict—precisely what an adaptive adversary has.

The lesson is not that these particular heuristics are weak; it is that *oracle access to any fixed decider suffices to defeat it.* A more powerful agent raises the attacker's constant factor, not the asymptotics.

6 Study 3: Pricing the Trade-off Under a Real Adversary

Soundness-with-abstention (Study 1) and forced-decision fragility (Study 2) are the two theoretical poles. Real detonation/sandbox pipelines live in between: they run a sample for a bounded *analysis window* and flag it if malice manifests. The adversary's countermeasure is timing—logic bombs, long sleeps, environment-triggered payloads that simply outlast the window.

We model a population of 20,000 programs, 40.0% genuinely benign (they never trigger) and the rest malicious with a trigger delay drawn from an explicit heavy-tailed mixture (quick,

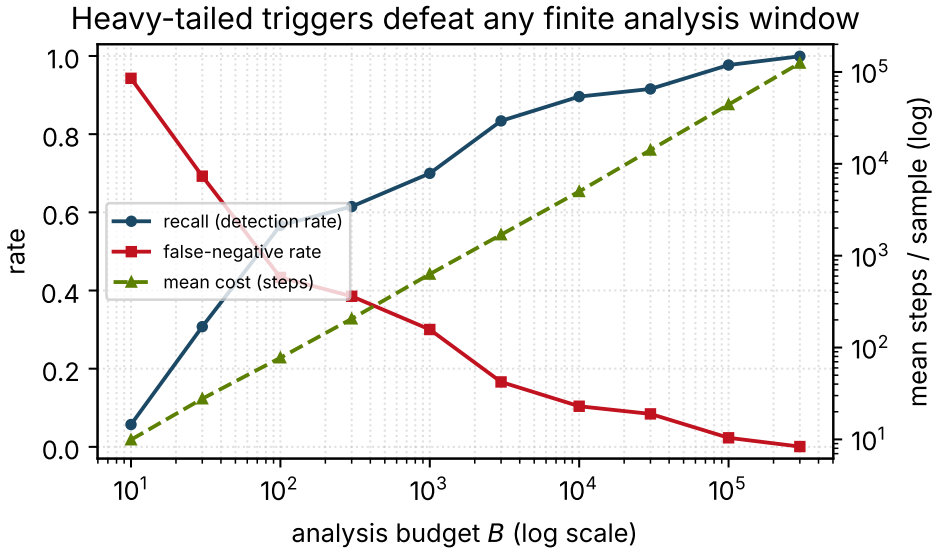


Figure 2: Recall rises with the analysis window but the heavy tail of trigger delays makes the final stretch brutally expensive: closing the last few percent of false negatives costs orders of magnitude more compute, and a dormant fraction outlasts any finite window.

delayed, and dormant components; median delay 63 steps, 99th percentile 147,488 steps). A sound window of size B catches a malicious sample iff its delay is $\leq B$; benign samples are never falsely flagged. Figure 2 shows recall, false negatives, and mean cost as B grows.

At a realistic 1,000-step window the detector achieves 70.0% recall—so 30.0% of malicious samples slip through purely by waiting. No window size removes the tail: this is the budgeted, probabilistic shadow of Proposition 1 and of the NP-completeness of bounded detection [5]. An agent that reports "70.0% caught" as "safe" is mistaking its budget for a proof.

7 Discussion: Designing Agents That Know Their Limits

The results are negative about *completeness*, not about usefulness. They suggest a design posture.

- **Prefer soundness with explicit abstention.** Study 1 says an agent can be trustworthy about what it *does* flag while being honest that silence is not safety. A third verdict—UNDETERMINED—is not a cop-out; it is the only truthful output for the residue.
- **Treat adaptive evasion as structural.** Study 2 says any fixed decider is defeatable by an adversary with oracle access. Defence should assume the attacker can query the agent and should diversify and randomise rather than trust a single frozen policy.
- **Budget is not proof.** Study 3 says recall figures are statements about an assumed delay distribution, not guarantees. Agents should publish their analysis budget and the evasion cost it implies, so a 70.0% number is read as "under this budget and this threat model".
- **Compose undecidable detection with decidable containment.** Because the verdict problem is undecidable, security should not rest on it alone: least privilege, capability confinement, and reversible actions turn a missed detection into a bounded loss rather than a breach.

Scope and limitations. Study 1's population is random Turing machines, not real software, and random TMs halt early or not at all; the measured residue is an instance of the asymmetry, not an estimate of real-world base rates. Study 3's numbers are properties of a chosen distribution and should not be quoted as empirical malware statistics. Study 2 is a constructive impossibility result; it says nothing about how *often* real attackers build such evaders, only that they always can. None of the three weakens the central claim, which is a statement about computation, not about any particular dataset.

8 Related Work

The undecidability of detection traces to Turing [1] and Rice [2], applied to malicious code by Cohen [3] and sharpened by Chess and White [4]; Spinellis [5] established NP-completeness of bounded-length virus identification. A parallel literature studies the fragility of learned detectors under adaptation: Barreno et al. [7] framed the security of machine learning, and Biggio and Roli [8] traced a decade of evasion and poisoning attacks; Goodfellow et al. [9] showed the ease of adversarial examples. Our Study 2 connects that fragility back to its computability-theoretic root: adaptive evasion is not merely empirically easy, it is a diagonal argument. Sipser [6] is our standard reference for the underlying theory.

9 Conclusion

Autonomous AI security agents are bounded deciders for undecidable problems. We measured what that costs: a sound agent leaves an 81.5% uncertifiable residue it can never close (Study 1); any forced decider is defeated with 100.0% reliability by a diagonal evader (Study 2); and a realistic analysis window misses 30.0% of malicious samples to timing alone (Study 3). The engineering response is not a better oracle—none exists. It is to build agents that are sound, calibrated, and explicit about their blind spots, and to pair undecidable detection with decidable containment. An agent that knows what it cannot know is more secure than one that pretends to know everything.

References

[1] A. M. Turing, "On Computable Numbers, with an Application to the Entscheidungsproblem," *Proceedings of the London Mathematical Society*, ser. 2, vol. 42, pp. 230–265, 1936.

[2] H. G. Rice, "Classes of Recursively Enumerable Sets and Their Decision Problems," *Transactions of the American Mathematical Society*, vol. 74, no. 2, pp. 358–366, 1953.

[3] F. Cohen, "Computer Viruses: Theory and Experiments," *Computers & Security*, vol. 6, no. 1, pp. 22–35, 1987.

[4] D. M. Chess and S. R. White, "An Undetectable Computer Virus," in *Proceedings of the 2000 Virus Bulletin International Conference (VB2000)*, Orlando, FL, 2000.

[5] D. Spinellis, "Reliable Identification of Bounded-Length Viruses is NP-Complete," *IEEE Transactions on Information Theory*, vol. 49, no. 1, pp. 280–284, 2003.

[6] M. Sipser, *Introduction to the Theory of Computation*, 3rd ed. Boston, MA: Cengage Learning, 2013.

[7] M. Barreno, B. Nelson, A. D. Joseph, and J. D. Tygar, "The Security of Machine Learning," *Machine Learning*, vol. 81, no. 2, pp. 121–148, 2010.

[8] B. Biggio and F. Roli, "Wild Patterns: Ten Years After the Rise of Adversarial Machine Learning," *Pattern Recognition*, vol. 84, pp. 317–331, 2018.

[9] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and Harnessing Adversarial Examples," in *International Conference on Learning Representations (ICLR)*, 2015.